UNIVERSITÀ DI PISA

University of Pisa

Competitive Programming and Contests

Academic Year 2025–2026

HANDS-ON REPORT

Solution to Hands-On 2

Problem #1: Min and Max

Problem #2: Is There

This report presents the solutions for the second hands-on assignment of the course. The work focuses on building, querying, and updating Segment Trees in Rust. The first problem combines range `min` updates with range maximum queries. The second problem builds a coverage Segment Tree from a set of intervals and answers whether a queried range contains a point covered by exactly `k` intervals.

Name Oleksiy Nedobiychuk
Matricola 597455
Course Competitive Programming and Contests
Date July 13, 2026

Department of Computer Science, University of Pisa

Introduction

Here, I present the solutions to the two problems of Hands-On 2. Both solutions are implemented with Segment Trees, because the operations are performed on intervals and require faster answers than scanning the whole array for every query.

The implementations use a recursive Segment Tree stored in a vector. Instead of the classic heap layout with children at positions `2i+1` and `2i+2`, the code stores nodes in preorder: the left child of node `i` is `i+1`, while the right child is computed from the size of the left subtree. This makes it possible to store the tree in about $2N$ nodes.

Exercise 1: Min and Max

The first solution is implemented in `hands_on21.rs`. The input starts with the array, and then each following line is an operation. Operation `1 l r` asks for the maximum value in the interval $[l, r]$, while operation `0 l r v` updates every element in $[l, r]$ to the minimum between its current value and `v`. The public methods convert the one-based input indices to zero-based indices before accessing the tree.

Each Segment Tree node stores the maximum value of its interval. The build procedure initializes the leaves from the input array and then combines two children with `max`. A range maximum query follows the standard Segment Tree logic: if the current node interval is fully inside the query range, its stored value is returned immediately; if it is disjoint, the neutral value `0` is returned; otherwise the query is split between the two children and the maximum of the two answers is returned.

The update operation applies the assignment

$$a_i \leftarrow \min(a_i, v)$$

to all leaves in the update range. After a leaf is changed, the recursion recomputes the maximum stored in every ancestor on the path back to the root. This implementation does not use lazy propagation for the update, because the stored aggregate is a maximum while the update is a range `min` operation on individual values. Therefore, the update descends to the affected leaves and rebuilds the touched internal nodes.

Building the tree takes $O(N)$ time and $O(N)$ memory. A maximum query takes $O(\log N)$ time on the usual Segment Tree decomposition. An update touches all affected leaves, so its cost is proportional to the length of the updated interval, with a worst-case bound of $O(N)$.

Exercise 2: Is There

The second solution is implemented in `hands_on22.rs`. The input contains `n` segments and `m` queries. Each segment $[s, e]$ adds one unit of coverage to every position in that interval. Each query `l r k` asks whether there is at least one position in $[l, r]$ whose coverage count is exactly `k`. The program prints `1` when such a position exists and `0` otherwise.

For this problem, each Segment Tree node stores two values: the minimum and the maximum coverage count inside its interval. The tree is built by starting from zero coverage everywhere and inserting every segment as a range increment by one. Unlike Exercise 1, this solution uses lazy propagation: when an entire node interval is covered by a segment, both its minimum and maximum are incremented and a pending increment is recorded in the `lazy` array. Before descending into a node's children, the pending increment is pushed down so that the children contain up-to-date values.

To answer a query, the algorithm recursively visits the relevant parts of the Segment Tree. For a node fully contained in the query interval, it checks the stored pair `[min, max]`. If `k` lies between these two

values, the implementation returns `1`; otherwise that node cannot contain a position with coverage `k`, and it returns `0`. For partial overlaps, the query is split between the two children, and the final answer is the maximum of the two Boolean results.

If there are N positions and S input segments, building the coverage tree takes $O(S \log N)$ time, because every segment is inserted as one lazy range increment. The memory usage is $O(N)$. Each query visits a logarithmic number of Segment Tree nodes in the standard case, so its expected cost is $O(\log N)$.

Testing

The implementations were validated with the provided input and output files. The first binary was checked directly against the files in `data/test_handson21`. The second binary was checked with `helpers/h22.sh` against `data/test_handson22`. In both cases, the produced output was compared with the expected output using `diff`.

Overall, these tests check the main behavior of both Segment Tree implementations: correct aggregation after updates in the first problem, and correct lazy propagation and range querying in the second problem.